Segurança de Dados

MCTA023-17

Programação Segura - Parte 1

Buffer Overflow e String de Formatação

Profª Denise Goya

Roteiro

Aspectos de Segurança no Software e no Sistema Operacional:

- Buffer Overflow
- String de Formatação

Programação Segura

Objetivos

- Entender como são as fontes de insegurança
- Conhecer **vulnerabilidades** que fazem a programação menos segura e como devemos evitá-las
- Conhecer princípios, métodos e tecnologias que fazem a programação mais segura

Exemplos de Vulnerabilidade

Para ter uma ideia dos problemas em software e para se manter atualizado, caso você seja administrador de TI/redes:

http://www.securityfocus.com/vulnerabilities

http://www.us-cert.gov/cas/bulletins

http://www.securitytracker.com/

nvd.nist.gov/vuln/detail/CVE-2020-11500

NIST

Information Technology Laboratory

NATIONAL VULNERABILITY DATABASE

Vulnerabilidades: escolhas inadequadas de configurações criptográficas

VULNERABILITIES

CVE-2020-11500 Detail

Current Description

Zoom Client for Meetings through 4.6.9 uses the ECB mode of AES for video and audio encryption. Within a meeting, all participants use a single 128-bit key.

+View Analysis Description

Severity

CVSS Version 3.x | CVSS Version 2.0

CVSS 3.x Severity and Metrics:

NIST: NVD **Base Score:** 7.5 HIGH **Vector:** CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

QUICK INFO

CVE Dictionary Entry:
CVE-2020-11500
NVD Published Date:
04/03/2020
NVD Last Modified:
04/07/2020
Source:
MITRE

Exemplo de Vulnerabilidade

Roteador Cisco (source US-CERT)

Publicado: 2011-01-24

Vulnerabilidade número: CVE-2011-0352

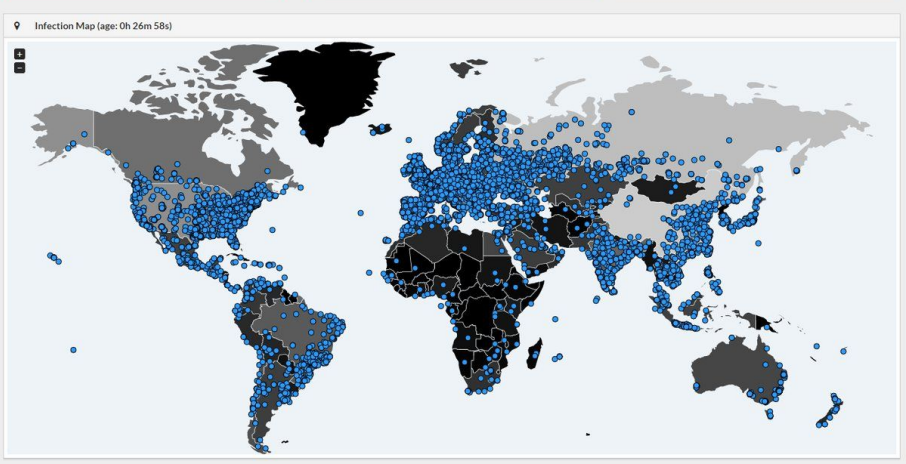
CVSS Severity Score: 7.88

Produto: cisco – linksys_wrt54gc_router_firmware

Buffer overflow in the web-based management interface on the Cisco Linksys WRT54GC router with firmware before 1.06.1 allows remote attackers to cause a denial of service (device crash) via a long string in a POST request.

Atualizações de Segurança (patches)

- Correções de segurança devem ser instaladas
- Sistemas antigos devem ser substituídos na medida do possível
- Caso: ransonware WannaCry, em 2017, que explora vulnerabilidade conhecida em MS-Windows, mas versões antigas não foram corrigidas



Fonte: https://cyberwar.liveuamap.com/en/2017/14-may-past-24-hours-wannacry-infection-map

Programação Segura e Norma ISO 27002

- 12. Aquisição, Desenvolvimento e Manutenção de Sistemas de Informação
  - 12.2 Processamento Correto nas Aplicações
    - 12.2.1. Validação dos Dados de Entrada
    - Controle: Convém que os dados de entrada de aplicações sejam validados para garantir que são corretos e apropriados
    - 12.2.2. Controle do processamento interno
    - Controle: Convém que sejam incorporadas nas aplicações, verificações de validação com o objetivo de detectar qualquer corrupção de informações, por erros ou por ações deliberadas
    - 12.2.3. Integridade de mensagens
    - 12.2.4. Validação de dados de saída

Questões de Insegurança

- Erros comuns:
  - Buffer Overflow
  - Strings de Formatação
  - Injeção de Código e Validação de Entrada
  - Cross-site Scripting (XSS)
  - E vários outros

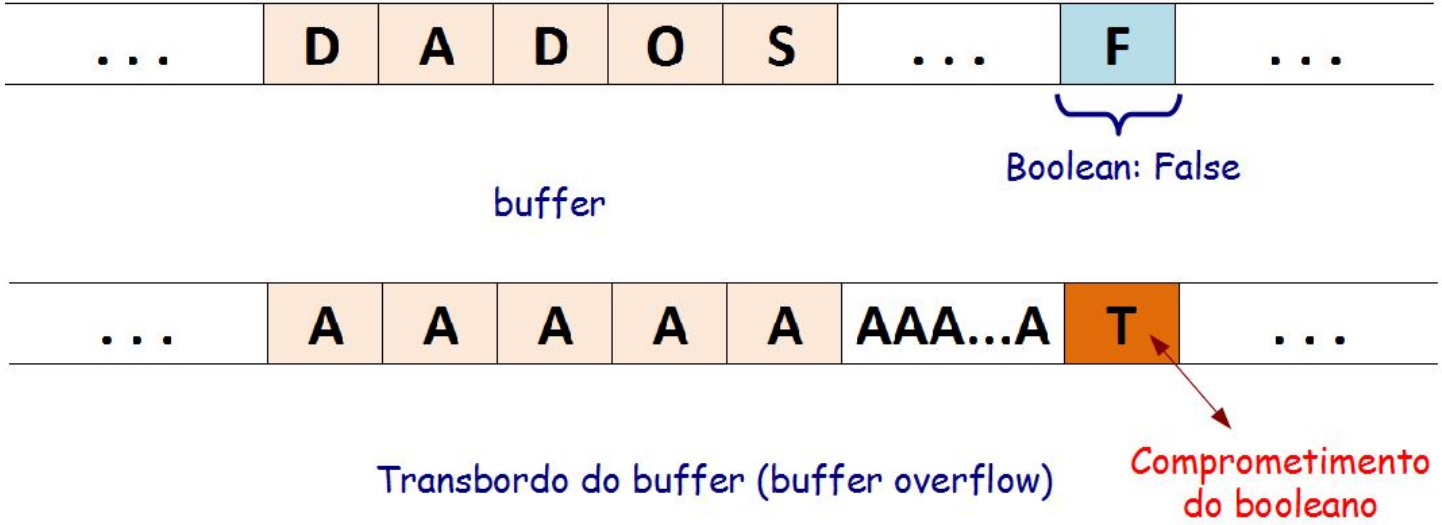
Buffer Overflow

Buffer Overflow

- Transbordo de memória
- Tipo mais famoso e muito explorado de vulnerabilidade
- Causa:
  - O programador não limita a quantidade de informação que pode ser escrita em uma determinada área de memória (buffer: string, array, etc)

Buffer Overflow: exemplo

- Variáveis na memória:
  - Array de 5 posições (ex: char buffer[5])
  - Variável booleana (true/false)

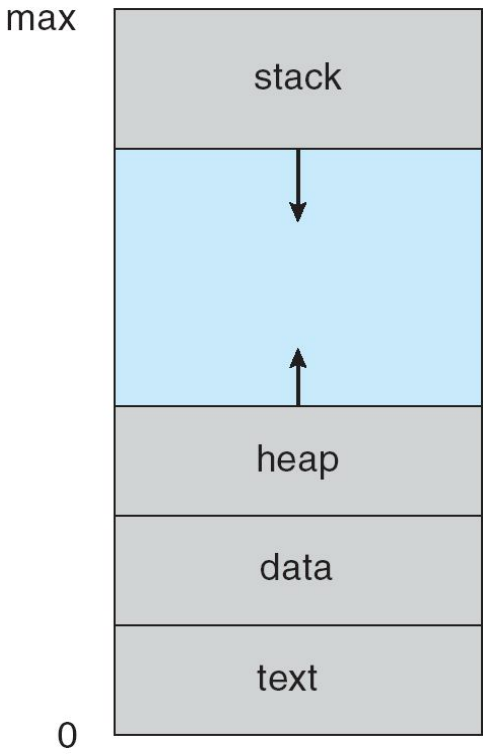


Buffer Overflow - SO

- Para entender como o buffer overflow pode ser explorado em ataque contra a segurança:
  - Veremos, resumidamente, como o Sistema Operacional organiza a memória internamente
  - Algumas particularidades de implementação são dependentes do SO

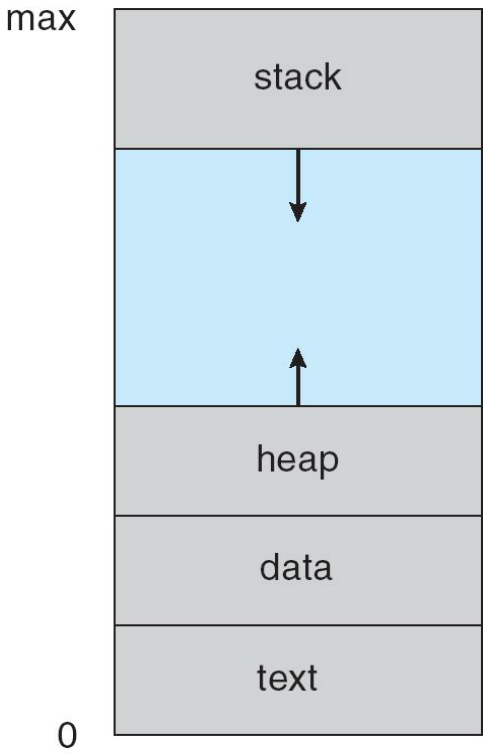
Memória

- O sistema operacional cria um processo que aloca memória e outros recursos
- Grandes componentes da memória (organização lógica):
  - Stack
  - Heap
  - Dados
  - Códigos (área text)



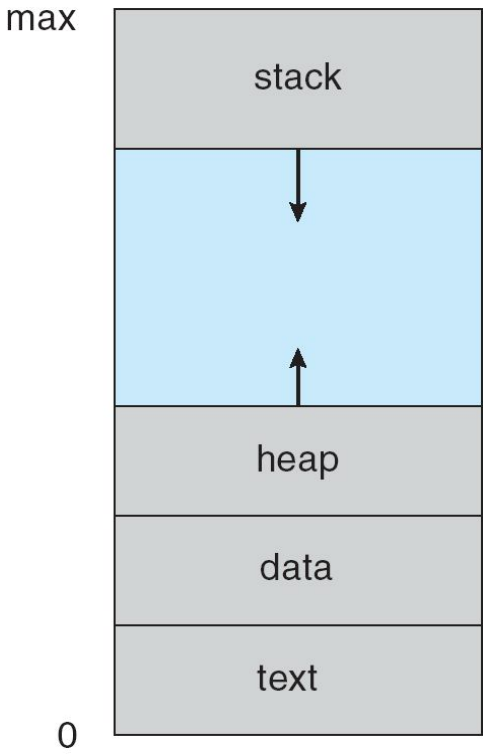
Text (Códigos)

- Área com os códigos dos processos do usuário, as instruções do programa que serão executadas



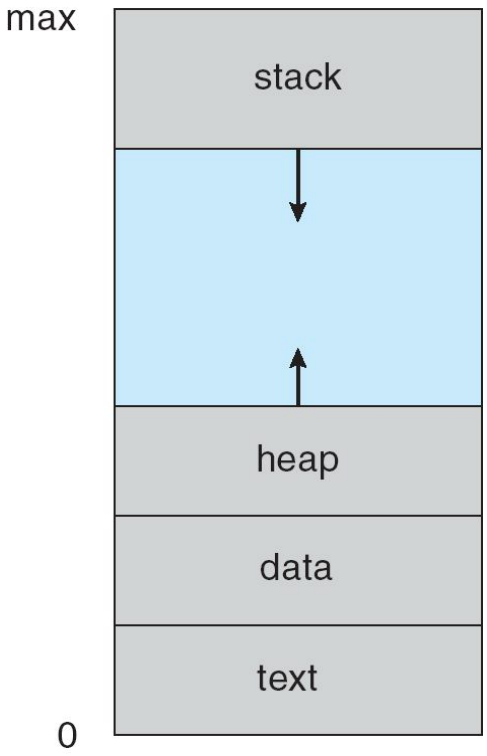
Data (Dados)

- Área em que são armazenadas as variáveis, incluindo variáveis **globais e estáticas**, ou não inicializadas



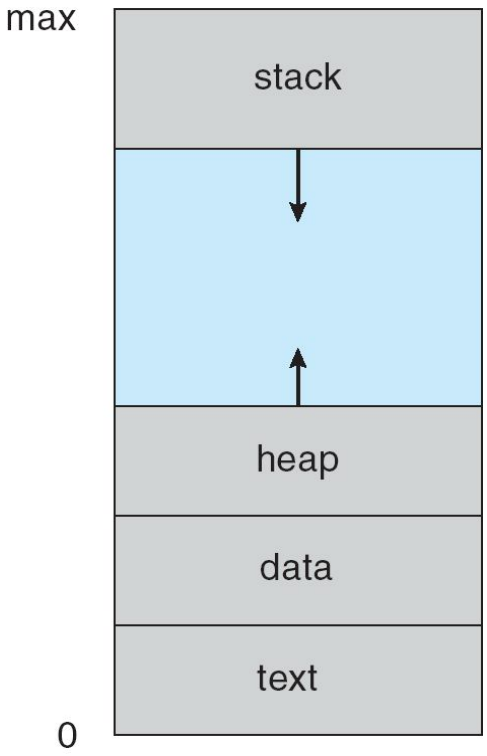
Stack (pilha de execução)

- guarda o ponto em que cada subrotina ativa deve retornar o controle quando terminar a execução;
- **armazena as variáveis que são locais** para as funções e seus argumentos (parâmetros);



Heap

- memória dinâmica para variáveis criadas com **malloc**, calloc, realloc e liberadas com free



Processo na Memória

- Chamamos de **Processo** um programa em execução
- Veremos, resumidamente, como é o controle de execução de um processo na memória

Contador de Programa – PC

- O contador de programa (program counter – PC) aponta para a próxima instrução a ser executada (ou a instrução em execução)

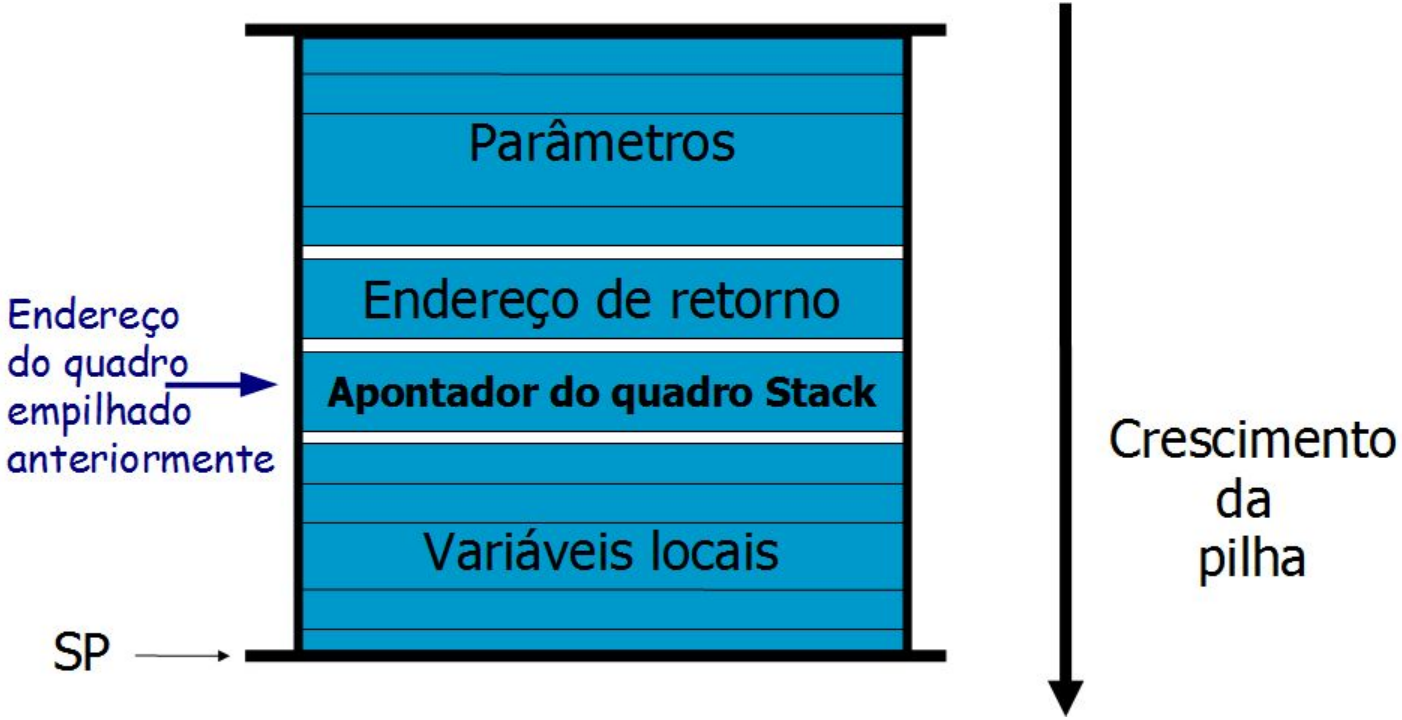
Chamada de Função

- Procedimento de chamada de função:
  - Prepara os parâmetros
  - Salva o estado e aloca na pilha as variáveis locais
  - Aponta para o começo do procedimento solicitado

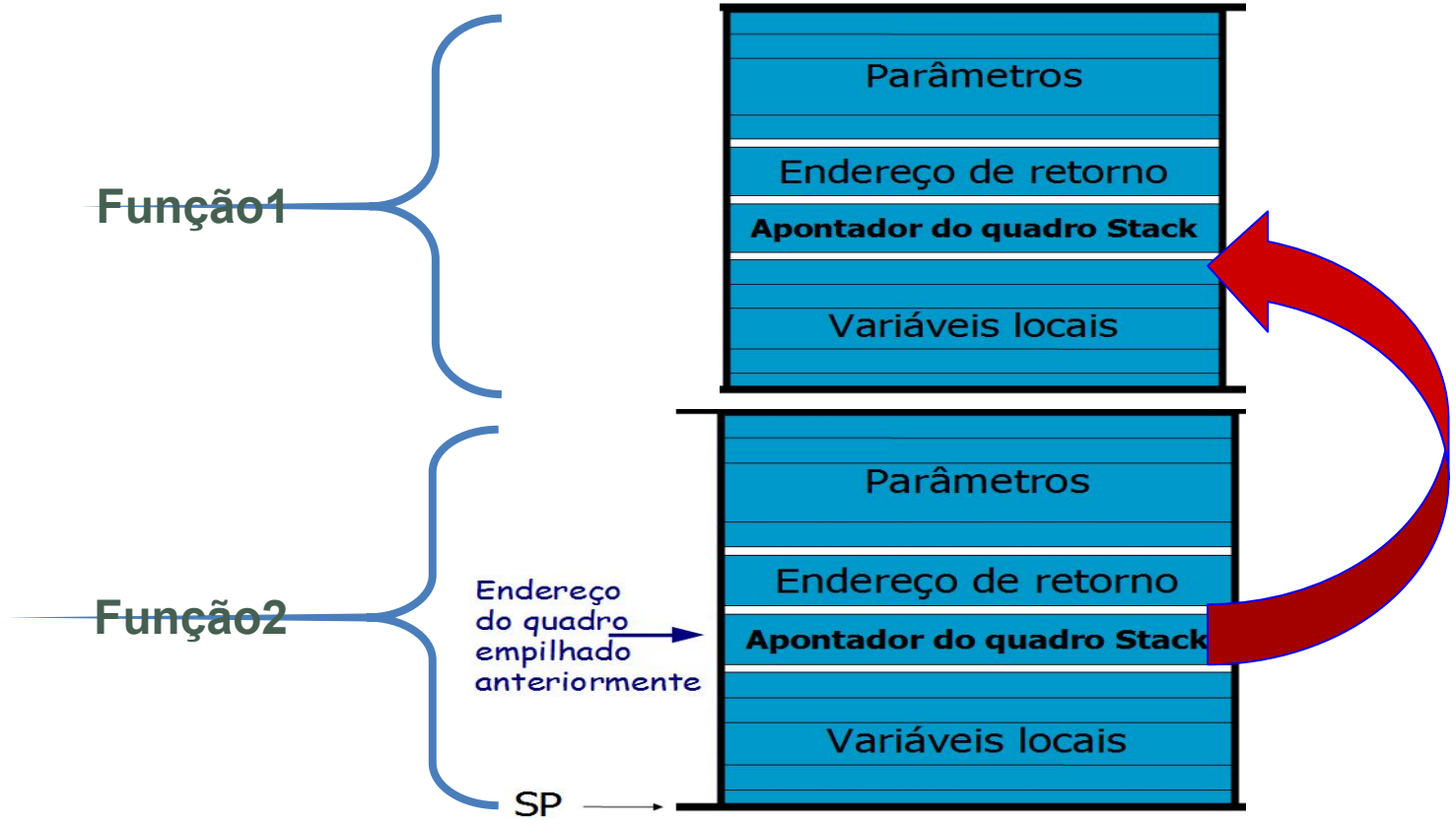
Retorno de uma Função Chamada

- Procedimento de retorno:
  - Grava o estado atual da pilha e ajusta a pilha
  - Execução continua a partir do endereço retornado

Pilha de Execução



Pilha de Execução

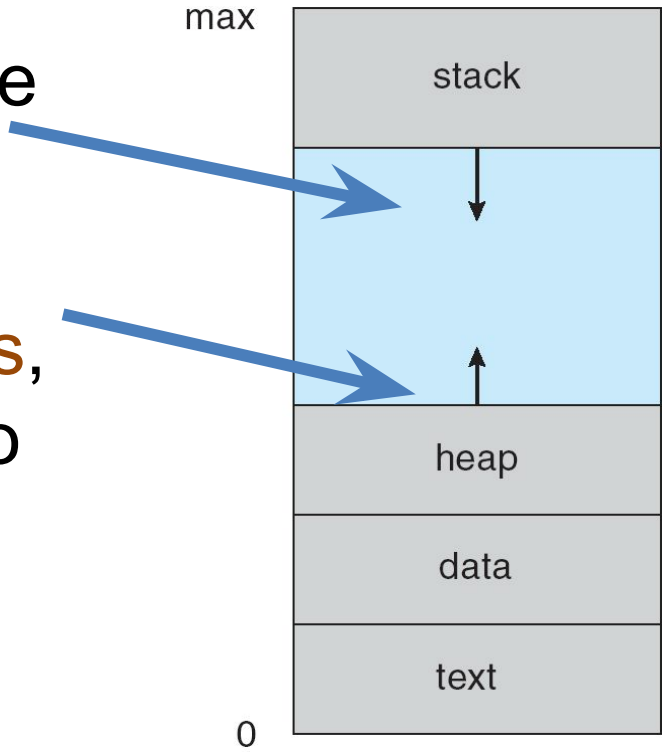


Parâmetros na Pilha

- Passos para tratar os parâmetros:
  - Salva o PC na stack (endereço de retorno)
  - Salva o valor de SP na stack (função empilhada anteriormente)
  - Aloca o espaço da stack para variáveis locais

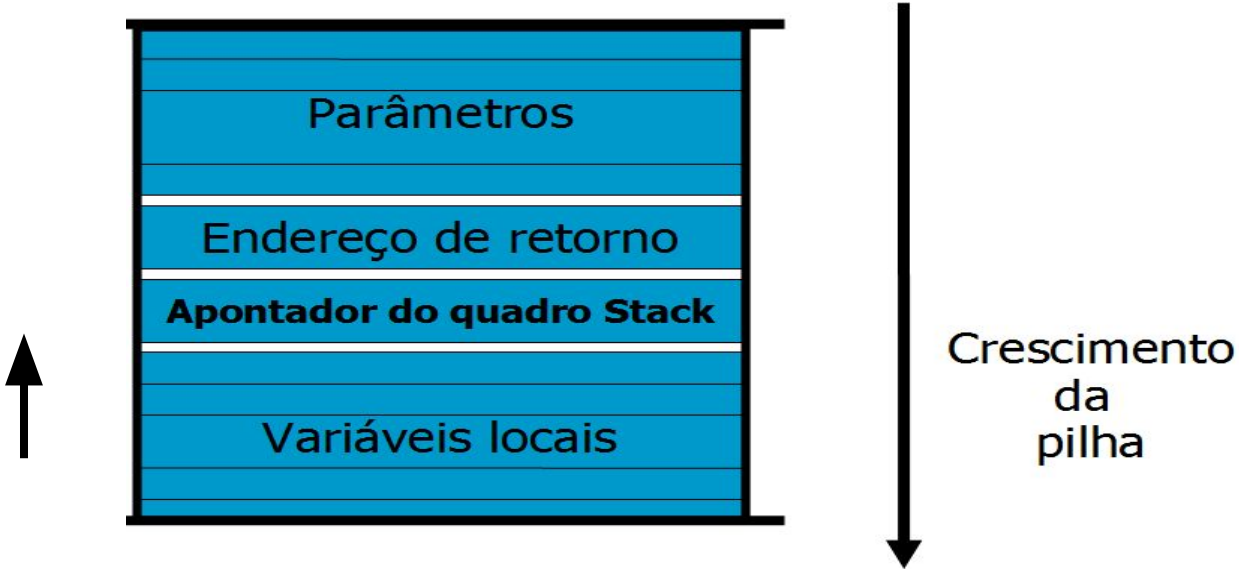
Ordem de Alocação

- A pilha de execução cresce num sentido,
- enquanto os demais dados, crescem no sentido oposto



Ordem de Alocação

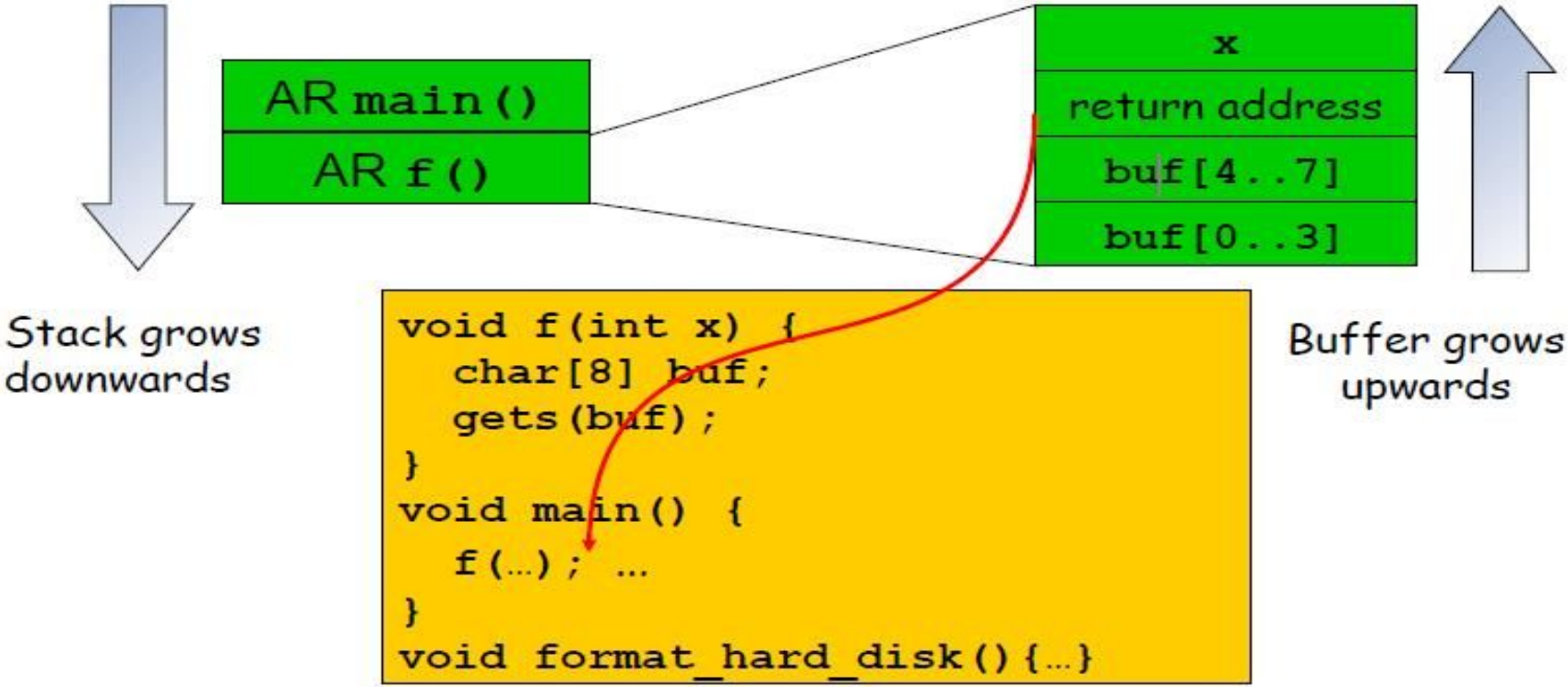
- A pilha de execução cresce num sentido, enquanto os dados locais à função, crescem no sentido oposto



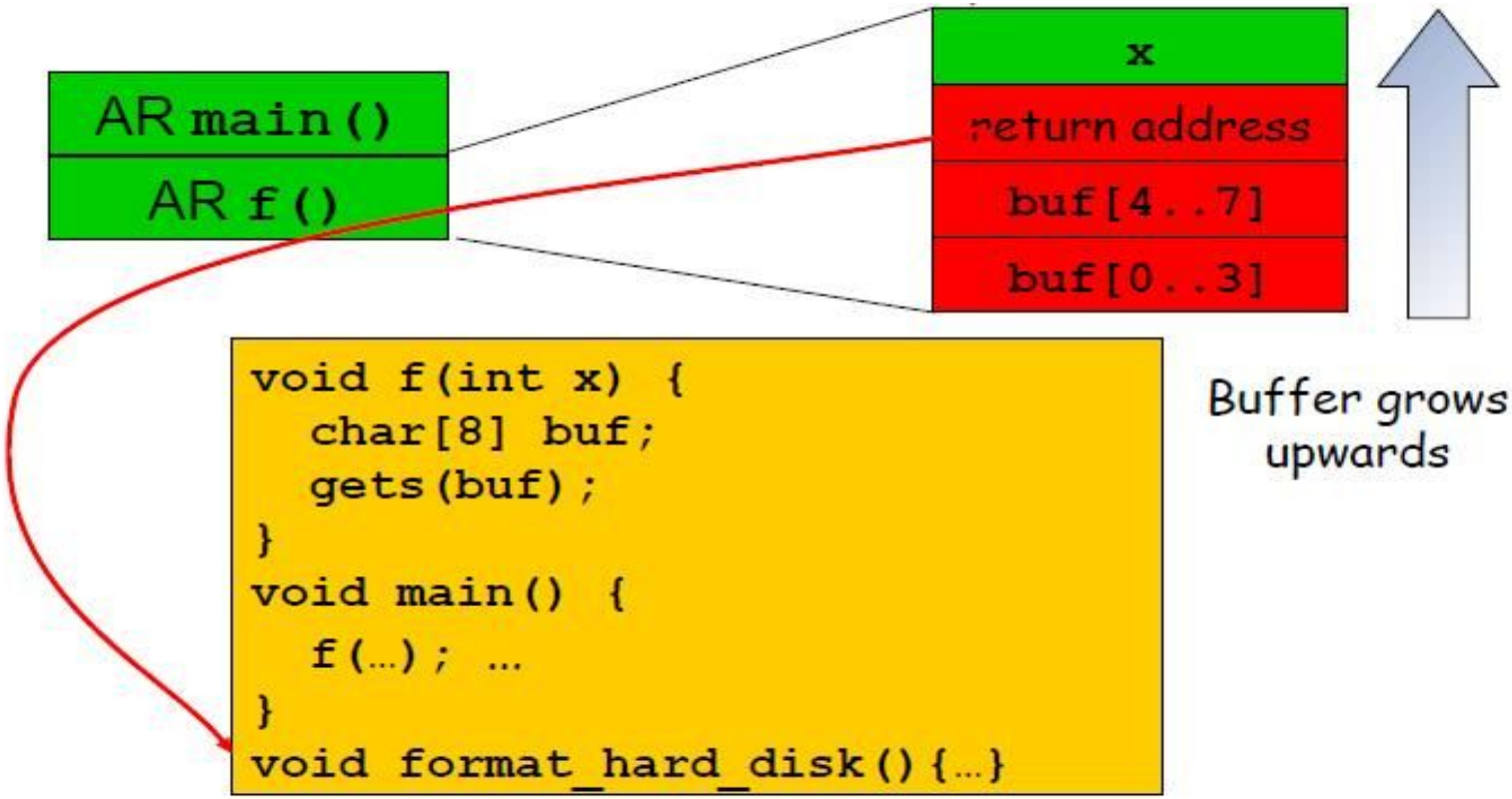
17/mar.	Programação segura (buffer overflow, string de formatação)	19/mar.	T	Programação segura (SQL Injection, XXS)
24/mar.	Apresentação de trabalhos (Criptografia pós-quântica e Firewall)	26/mar.	L	Prática: segurança no software
31/mar.	Apresentação de trabalhos (malware, segurança em SO específicos e em aplicações Web)	2/abr.	T	Apresentação de trabalhos (DDoS, Desenvolvimento Seguro de Aplicações Web, LGPD)
7/abr.	Apresentação de trabalhos (segurança em SO, Blockchain, proteção LLM)	9/abr.	L	**Prova P1**
14/	Apresentação de trabalhos (Normas, Defesa cibernética e Segurança em BD)	16/abr.	T	Apresentação de trabalhos (SHA3, desenvolvimento seguro de software, IDS)
r	Feriado de Tiradentes	23/abr.	L	**Prova P2 (conteúdos dos seminários)**

G20: tem que apresentar em 7/abr. (não pode apresentar no dia da prova)

Buffer Overflow: Dinâmica



Buffer Overflow: Dinâmica



Buffer Overflow

- Pode ocorrer tanto na pilha quanto no heap
- Estouro de stack:
  - é estouro de um buffer alocado na pilha (conhecido como **stack smashing** ou **stack overflow**)
- Estouro de heap:
  - estouro do buffer alocado no heap

Funções em C Problemáticas

- Algumas funções em ANSI-C podem ser fonte de buffer overflow quando mal usadas:
  - strcpy (char *dest, const char *src)
  - strcat (char *dest, const char *src)
  - gets (char *s)
  - scanf (const char *format, …)
  - printf (conts char *format, …)
  - . . .

Disseminação da Linguagem C

- A linguagem C é largamente usada em
  - Sistemas Operacionais
  - funções de baixo nível mesmo em outras linguagens de programação
  - codificação de drivers (controladores de dispositivos de hardware)
- Aplicações (no alto nível) fazem acesso a funções mais elementares (muitas vezes codificadas em C)

Exemplos de Funções de Biblioteca C

- **gets**

```
char buf[20];
gets(buf);    // lê a entrada do usuário até
              // primeiro caractere EoF
```

- Nunca use gets
- Use **fgets(buf, size, stdin)**

Exemplos de Funções de Biblioteca C

- **strcpy**

```
char dest[20];
strcpy(dest, src); // copia string src para dest
```

- strcpy assume: que *dest* é longo o bastante e que *src* possui um **'\0'** que encerra a string
- Use **strncpy(dest, src, size)**

Exemplos de Funções de Biblioteca C

function

strcpy

<cstring>

```
char * strcpy ( char * destination, const char * source );
```

Copy string

Copies the C string pointed by *source* into the array pointed by *destination*, including the terminating null character (and stopping at that point).

To avoid overflows, the size of the array pointed by *destination* shall be long enough to contain the same C string as *source* (including the terminating null character), and should not overlap in memory with *source*.

function

strncpy

<cstring>

```
char * strncpy ( char * destination, const char * source, size_t num );
```

Copy characters from string

Copies the first *num* characters of *source* to *destination*. If the end of the *source* C string (which is signaled by a null-character) is found before *num* characters have been copied, *destination* is padded with zeros until a total of *num* characters have been written to it.

No null-character is implicitly appended at the end of *destination* if *source* is longer than *num*. Thus, in this case, *destination* shall not be considered a null terminated C string (reading it as such would overflow).

destination and *source* shall not overlap (see memmove for a safer alternative when overlapping).

Dangerous C system calls - Building secure software, J. Viega & G. McGraw, 2002

Extreme risk
- gets

High risk
- strcpy
- strcat
- sprintf
- scanf
- sscanf
- fscanf
- vfscanf
- vsscanf

High risk (cntd)
- streadd
- strecpy
- strtrns
- realpath
- syslog
- getenv
- getopt
- getopt_long
- getpass

Moderate risk
- getchar
- fgetc
- getc
- read
- bcopy

Low risk
- fgets
- memcpy
- snprintf
- strccpy
- strcadd
- strncpy
- strncat
- vsnprintf

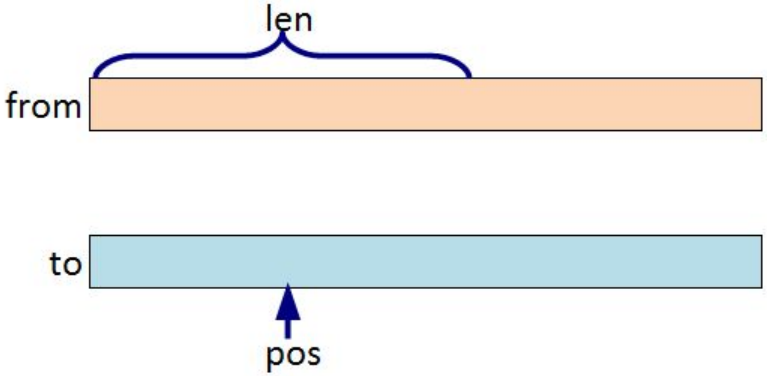
Buffer Overflow por Más Práticas

- No entanto, na prática, o problema maior **não** é causado pelas funções inseguras
  - os efeitos de Bibliotecas com rotinas problemáticas tendem a ser o menor dos problemas
- Más Práticas de Programação (Inseguras) são mais comuns

Buffer Overflow por Más Práticas

- Exemplo: por que o código abaixo é vulnerável?

```
int copy_buf (char *to, int pos, char *from, int len) {
int i;
  for (i=0; i<len; i++) {
    to[pos] = from[i];
    pos++;
  }
  return pos;
}
```



Buffer Overflow: Exploração

- Estouro do buffer pode ser explorado para abortar um programa ou serviço (ataque à **disponibilidade**)

- Mas também possibilita ataques mais sofisticados que permite total domínio de um sistema

Buffer Overflow com Injeção de Código

- É o caso de exploração de buffer overflow mais elaborado e potencialmente perigoso
- Sobrescreve a memória com **código** de interesse do atacante:
  - o endereço de retorno é modificado para uma região na **própria pilha** (ou heap), na qual o atacante colocou um código executável

Buffer Overflow: Shell code

- Código que dá acesso a uma **linha de comando** via chamada de sistema
  - Com os mesmos privilégios de acesso do programa explorado
  - Código tipicamente curto, com poucos bytes
- Consequências das mais diversas
- Grande poder ao atacante

Buffer Overflow: Prevenção

- Programação criteriosa
- Uso de bibliotecas e compiladores mais recentes
- Compilação com mecanismos de **proteção da memória** (pilha e heap)

Buffer Overflow: Prevenção

- Compilação com mecanismos de **proteção da pilha**:
  - Deteção de modificação do endereço de retorno de funções na pilha (**stackguard)**:
    - **Canário**: valor gravado na pilha para indicar modificações
  - Detecção de escritas indevidas na pilha (stackshield)
- Disponível em qualquer compilador mais recente

Buffer Overflow: Prevenção

- Proteção do espaço de endereçamento executável
  - Pode ser usada para impedir que código seja executado a partir do heap ou stack
  - Requer suporte da unidade de gerenciamento de memória (MMU) do processador
- Aleatorização do espaço de endereçamento
  - Aleatoriza os endereços de início de stack e heap
  - Torna mais difícil um atacante adivinhar os endereços para efetivar ataques

Buffer Overflow: Considerações Finais

- Vulnerabilidade cuja exploração é complexa, porém com muitos kits disponíveis livremente
- Prevenção requer o envolvimento de:
  - programadores em geral (sistemas baixo e alto nível)
  - desenvolvedores de compiladores
  - desenvolvedores de sistemas operacionais
  - fabricantes de processadores
  - arquitetos de computadores
- Ataques são viáveis e críticos em muitos casos
- Atualizações de segurança: instale-as

STRINGS DE FORMATAÇÃO

Strings de Formatação

printf (“uma string de formação %d %s ”, ...)

- Permite que um invasor passe como parâmetro especificadores de conversão (ex: “%d”, “%s”) e faça com que sejam processados mais dados do que o programador considerou originalmente
- Permite que endereços de memória sejam sobrescritos e código malicioso seja executado
- O atacante precisa ter muito conhecimento

Strings de Formatação

- Problema com linguagens C e C++
- Quantidade variável de argumentos em funções
  - argumentos faltando não são identificados
- O que vair ocorrer se o seguinte código for compilado e executado?

```
int main () {
    printf("Juliana tem %d gatos");
}
```

Strings de Formatação

- Resultado possível do programa:

  Juliana tem -1073742416 gatos

- O programa lê o argumento que está faltando da pilha do processo
- E pega lixo
  - ou coisas interessantes se o atacante deseja vasculhar a pilha

Strings de Formatação

- Digamos que o programador escreveu

```
int main () {
    printf (nomeUsuario);
}
```

  com a intenção de exibir uma variável do tipo String

- se nomeUsuario = "Juliana tem %d gatos"
- em tempo de execução, a função printf (a ser alocada na pilha de execução) é processada como

```
printf("Juliana tem %d gatos");
```

← e ocorre o erro

Strings de Formatação

Principais parâmetros de formatação:

- %d - decimal (int) – parâmetro por valor
- %u - unsigned decimal (unsigned int) – valor
- %x - hexadecimal (unsigned int) – valor

Os casos exploráveis por atacantes são:

- %s - string ((const) (unsigned) char *) - referência
- %n – número de bytes escritos até então (* int)- referência

Strings de Formatação

Ataque: Crashing

- printf ("%s%s%s%s%s%s%s%s%s%s%s%s");

- Para cada %s, printf() buscará um número na pilha, considerando esse número como endereço, e imprimirá o conteúdo da memória apontado por esse endereço como uma string até um caractere NULL encontrado

- Uma vez que o número carregado por printf() pode não ser um endereço, a memória aponta para esse número que pode não existir ou pertencer a área protegida e o programa irá falhar

Strings de Formatação

Ataques mais sofisticados

- Requer maior experiência em programação
- Pode-se ter acesso a certas áreas de memória
- Pode-se imprimir dados críticos (valor de chaves, por exemplo)
- Pode-se alterar dados de variáveis quaisquer

Strings de Formatação: Prevenção

- Ao usar a função printf, sempre deixe explícita a string de formatação:

```
int main () {
    printf ("%s", nomeUsuario);
}
```

- Use compiladores mais recentes, que em geral não compilam expressões como

```
printf (nomeUsuario);
```

- Garanta que os programas estejam sendo compilados com diretivas que detectem esses problemas

Próximas Aulas

- quinta-feira:
  - mais sobre programação segura
- Na outra semana:
  - Apresentações de trabalhos

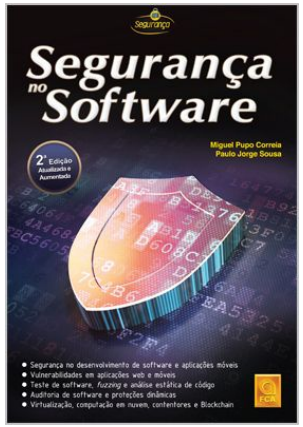
17/mar.	Programação segura (buffer overflow, string de formatação)	19/mar.	T	Programação segura (SQL Injection, XXS)
24/mar.	Apresentação de trabalhos (Criptografia pós-quântica e Firewall)	26/mar.	L	Prática: segurança no software
31/mar.	Apresentação de trabalhos (malware, segurança em SO específicos e em aplicações Web)	2/abr.	T	Apresentação de trabalhos (DDoS, Desenvolvimento Seguro de Aplicações Web, LGPD)
7/abr.	Apresentação de trabalhos (segurança em SO, Blockchain, proteção LLM)	9/abr.	L	**Prova P1**
14/	Apresentação de trabalhos (Normas, Defesa cibernética e Segurança em BD)	16/abr.	T	Apresentação de trabalhos (SHA3, desenvolvimento seguro de software, IDS)
r	Feriado de Tiradentes	23/abr.	L	**Prova P2 (conteúdos dos seminários)**

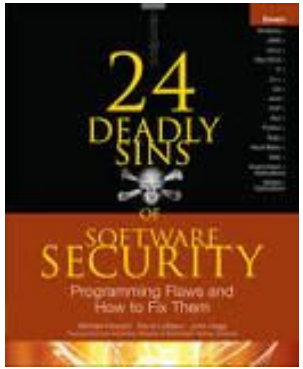
G20: tem que apresentar em 7/abr. (não pode apresentar no dia da prova)

Estudo Individual

Leitura Recomendada

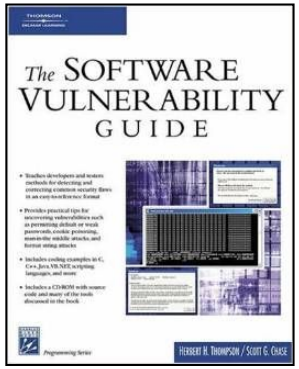
- CORREIA, M.P.; SOUSA, P.J.: "**Segurança no Software**" 2ª ed., FCA, 2017.
- HOWARD, M.; LEBLANC, D.; VIEGA, J.: "**24 Deadly Sins of Software Security: Programming Flaws and How to Fix Them**" McGraw-Hill Ed, 2009

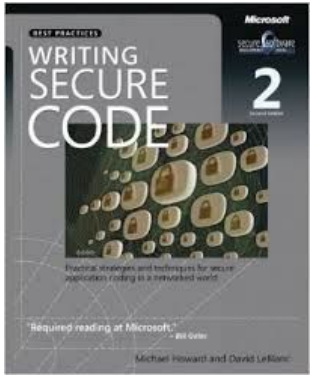




Leitura Recomendada

- THOMPSON, H.; CHASE, SCOTT G.: "**The Software Vulnerability Guide**" Charles River Media, 2005
- HOWARD, M.; LEBLANC, D.: "Writing Secure Code" Microsoft Press, 2a edição, 2002.
- WEELER, D.: "Secure Programming for Linux and Unix HOWTO – Creating Secure Software". Ebook disponível em http://www.dwheeler.com/secure-programs/





Questões de Autoavaliação

(não precisa entregar)

Exercício

Dos problemas estudados, cite quais podem afetar os requisitos de segurança abaixo e justifique:

- confidencialidade
- integridade
- disponibilidade